RAG Pipeline - Module Inputs & Outputs

Data contracts between modules for seamless integration.

---

load_docs.py

`load_pdfs_from_directory(directory_path: str)`

Input: `str` - PDF directory path

Output: `Tuple[List[str], List[Dict]]`

```python
# documents: List[str] - Raw text from PDFs
# metadata: List[Dict] - Document info
[
    {"source": str, "title": str, "length": int, "pages": int},
    {"source": str, "title": str, "length": int, "pages": int}
]
```

`adaptive_chunking(documents, document_metadata, ...)`

Input:

- `documents: List[str]` - Document texts
- `document_metadata: List[Dict]` - From load_pdfs_from_directory()

Output: `Tuple[List[str], List[int], List[Dict]]`

```python
# chunked_docs: List[str] - Text chunks
# doc_mapping: List[int] - [0, 0, 1, 1, 2] (which doc each chunk belongs to)
# chunk_metadata: List[Dict]
[
    {
        "original_doc_idx": int,
        "chunk_idx": int,
        "title": str,
        "source": str,
        "is_full_doc": bool
    }
]
```

🔍 retrieval.py

`DocumentRetriever.create_embeddings(chunked_docs, doc_mapping, chunk_metadata)`

Input:

- `chunked_docs: List[str]` - From adaptive_chunking()
- `doc_mapping: List[int]` - From adaptive_chunking()
- `chunk_metadata: List[Dict]` - From adaptive_chunking()

Output: None (stores internally)

`DocumentRetriever.retrieve_relevant_docs(query: str, k: int)`

Input:

- `query: str` - User question
- `k: int` - Number of chunks to retrieve

Output: `Tuple[List[Dict], float]`

python

```
# results: List[Dict]
[
    {
        "index": int,           # Chunk position in original list
        "score": float,         # 0.0-1.0 similarity score
        "content": str,         # Actual chunk text
        "original_doc_idx": int,# Which document this came from
        "metadata": Dict        # Same as chunk_metadata from load_docs
    }
]
# retrieval_time: float - Seconds taken
```

`DocumentRetriever.process_retrieved_chunks(retrieved_chunks, max_chunks_per_doc)`

Input: `retrieved_chunks: List[Dict]` - From retrieve_relevant_docs()

Output: `List[Dict]` - Same format as input but merged/filtered

🔗 augmentation.py

`create_augmented_prompt(query: str, retrieved_docs: List[Dict])`

Input:

- `query: str` - User question
- `retrieved_docs: List[Dict]` - From retrieval.py (processed chunks)

Output: `Tuple[str, float]`

```python
# prompt: str - Complete formatted prompt for LLM
# prompt_time: float - Seconds taken
```

`create_chat_messages(query: str, context: str)`

Input:

- `query: str` - User question
- `context: str` - Formatted context with citations

Output: `List[Dict[str, str]]`

```python
[
    {"role": "system", "content": "system instructions"},
    {"role": "user", "content": "question with context"}
]
```

`format_context_with_citations(retrieved_docs: List[Dict])`

Input: `retrieved_docs: List[Dict]` - From retrieval.py

Output: `str` - "[1] From title: content\n[2] From title: content"

🤖 generation.py

`AnswerGenerator.generate_answer_gpt4(prompt: str, ...)`

Input: `prompt: str` - From augmentation.py

Output: `Tuple[str, float]`

```python
# answer: str - Generated response
# generation_time: float - Seconds taken
```

`AnswerGenerator.generate_answer_with_messages(messages: List[Dict], ...)`

Input: `messages: List[Dict]` - From augmentation.py (chat format)

Output: `Tuple[str, float]` - Same as generate_answer_gpt4()

🔧 rag_pipeline.py

`RAGPipeline.query(query: str, k: int, use_chat_format: bool)`

Input:

- `query: str` - User question
- `k: int` - Number of chunks to retrieve
- `use_chat_format: bool` - Use chat vs simple prompt

Output: `Dict[str, Any]`

python

```
{
    "query": str,                    # Original question
    "retrieved_docs": List[Dict],    # From retrieval.py
    "processed_docs": List[Dict],    # Merged chunks
    "answer": str,                   # Final generated answer
    "metrics": {
        "retrieval_time": float,
        "processing_time": float,
        "prompt_time": float,
        "generation_time": float,
        "total_time": float
    }
}
```

🔄 Data Flow Chain

```
PDFs → load_docs → documents: List[str] + metadata: List[Dict]
                        ↓
        adaptive_chunking → chunks: List[str] + mappings: List[int] + chunk_meta: List[Dict]
                        ↓
        retrieval → retrieved_docs: List[Dict] (with scores)
                        ↓
        augmentation → prompt: str OR messages: List[Dict]
                        ↓
        generation → answer: str + time: float
                        ↓
        pipeline → complete_result: Dict[str, Any]
```

🔧 Module Compatibility Rules

When modifying **load_docs.py**:

- Keep `documents: List[str]` format
- Maintain metadata dict keys: `source`, `title`, `length`, `pages`
- Preserve chunk_metadata dict keys: `original_doc_idx`, `title`, `source`

When modifying **retrieval.py**:

- Keep retrieved_docs format with `index`, `score`, `content`, `metadata`
- Maintain score as float 0.0-1.0
- Preserve `original_doc_idx` for document tracking

When modifying **augmentation.py**:

- Input must accept `List[Dict]` from retrieval
- Output must be `str` (prompt) or `List[Dict]` (messages)
- Maintain citation format for traceability

When modifying **generation.py**:

- Accept `str` or `List[Dict]` input formats
- Always return `Tuple[str, float]` (answer, time)
- Handle API errors gracefully

When modifying **rag_pipeline.py**:

- Maintain final result dict structure
- Keep all timing metrics

- Preserve `query`, `answer`, `retrieved_docs` keys

Use these data contracts to ensure module compatibility when making changes.